

Distributed Theory & Communication

The rest of Part IV (consistency, consensus, time, transactions) and the start of Part V (APIs).

- 21 Consistency Models** — from strong to eventual, precisely defined
- 22 Consensus: Paxos & Raft** — getting a group to agree despite failures
- 23 Time, Clocks & Ordering** — ordering events without a global clock
- 24 Distributed Transactions & Coordination** — 2PC, locks, and why we avoid them
- 25 REST & API Design** — the web's dominant API style (opens Part V)

Chapters 21-24 complete the theory of distributed systems; Chapter 25 begins how services and clients communicate.

CHAPTER 21

Consistency Models

Part IV · Distributed Systems Theory — making “consistency” precise.

LEARNING OBJECTIVES

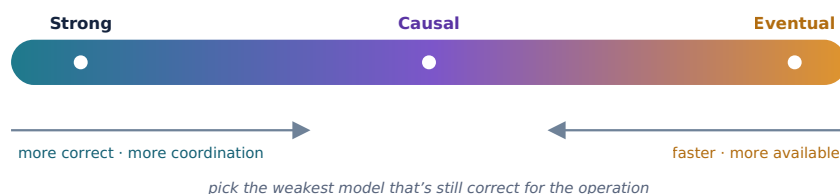
- Explain why “consistency” needs a precise definition across replicas.
- Define strong (linearizable) and eventual consistency and their costs.
- Explain the practical middle grounds: read-your-writes, monotonic reads, causal.
- Choose a consistency model per operation.

REAL-WORLD ANALOGY — CLERKS WITH COPIES OF A DOCUMENT

Several clerks each hold a copy of the same document. How current must each copy be when someone reads it? At one extreme, every copy must be identical the instant anyone edits (strong). At the other, copies just need to *agree eventually* once edits stop (eventual). Between those lie useful promises like “you always see your own edits.” Consistency models name these guarantees precisely.

21.1 The problem

Once data is replicated (Chapter 16) and CAP forces trade-offs (Chapter 20), the word “consistency” is too vague. A **consistency model** is a precise contract about what a read may return given prior writes across replicas — a spectrum from strict to relaxed.



21.2 Strong consistency (linearizability)

Every read returns the **most recent** write, and the system behaves as if there were a single copy updated in real time. Easy to reason about — but it requires coordination between replicas on every operation, adding latency and reducing availability during partitions (the CP choice). Use it where correctness is non-negotiable: balances, inventory, unique usernames.

21.3 Eventual consistency

Replicas are allowed to disagree temporarily; if writes stop, they **converge** to the same value. Reads may be **stale**. This is cheap, fast, and highly available (the AP choice) — ideal for feeds, view counts, DNS, and shopping carts, where a brief lag is harmless.

21.4 The practical middle grounds

Guarantee	Promise
Read-your-writes	You always see your <i>own</i> latest writes (fixes the “I posted but don’t see it” bug).
Monotonic reads	Successive reads never go <i>backward</i> in time.
Monotonic writes	Your writes are applied in the order you made them.
Causal	Causally-related writes are seen by everyone in the same order (reply never appears before its comment). The strongest model still highly available.

KEY INSIGHT — CAUSAL IS THE SWEET SPOT

Most “eventual consistency feels broken” bugs are really missing **read-your-writes** or **causal** ordering. Causal consistency preserves cause-and-effect (which relies on the ordering ideas of Chapter 23) while staying available — often the best practical default when strong consistency is too expensive.

21.5 Choosing a model

Stronger models are easier to reason about but cost coordination, latency, and availability. Pick the **weakest model that is still correct** for each operation — strong for money, causal or read-your-writes for user-facing interactions, eventual for counters and feeds. Many datastores let you tune this per request via quorum settings.

Common mistakes

WATCH OUT

- Using one blanket consistency level for the whole system instead of per operation.
- Shipping eventual consistency without **read-your-writes**, then fielding “my update vanished” reports.
- Paying for strong consistency on data that would be fine eventually consistent.
- Confusing ACID’s “C” (constraints) with the replication consistency discussed here.

Interview questions

1. Define strong vs eventual consistency and their costs.
2. What is read-your-writes, and what bug does it fix?
3. What is causal consistency, and why is it a good default?
4. How do you choose a consistency model per operation?

Hands-on exercises

1. For each, name a suitable model: bank balance, like count, user’s own profile edit, DNS.

2. Describe a user-visible failure from missing monotonic reads.
3. Explain how a quorum setting can move you along the spectrum.

Mini project

OBSERVE THE SPECTRUM

On a replicated store with tunable consistency, run write-then-read under replica lag at strong and at eventual consistency and record when the eventual read is stale. Then add a read-your-writes rule (route the user's own recent reads to the leader) and confirm the anomaly disappears for that user.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — A CONSISTENCY MAP

Label each feature with a consistency model: balances/settings → strong; a user's own posts → read-your-writes; comments/replies → causal; like/view counts and feed → eventual. This map refines the CP/AP decisions from Chapter 20 into concrete per-operation guarantees.

Chapter summary

- A **consistency model** precisely defines what a read may return given prior writes across replicas.
- **Strong** = always latest, costly; **eventual** = converges later, cheap and available.
- Middle grounds — **read-your-writes, monotonic reads, causal** — fix most real anomalies while staying available.
- Pick the **weakest correct** model per operation; tune via quorums.

CHAPTER 22

Consensus: Paxos & Raft

Part IV · Distributed Systems Theory — getting a group to agree despite failures.

LEARNING OBJECTIVES

- Explain what consensus is and where it's needed.
- Explain why a majority quorum gives agreement and fault tolerance.
- Describe how Raft does leader election and log replication.
- Name real systems built on consensus.

REAL-WORLD ANALOGY — A COMMITTEE REACHING ONE DECISION

A committee must agree on a *single* decision even though some members are absent and messages get lost. The rule that makes this safe: a decision counts only with a **majority**. Any two majorities share at least one member, so there can never be two conflicting decisions. That overlap is the heart of consensus.

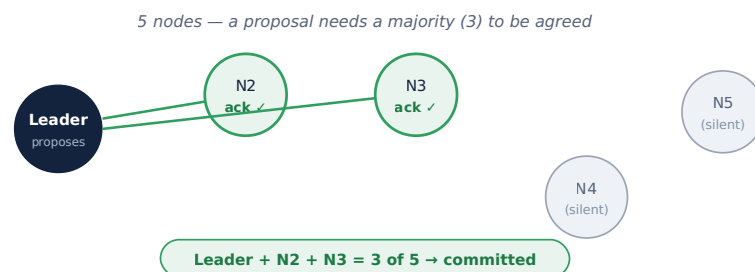
22.1 The problem

Distributed nodes must sometimes **agree on one value** — who is the leader, the order of entries in a log, a configuration change — despite crashes and lost messages (Chapter 19). **Consensus** is a protocol that gets a group to agree on a single value reliably, and it underlies most coordination in distributed systems.

22.2 Where consensus is needed

Leader election (Chapter 16 failover), **replicated logs** (state-machine replication — every replica applies the same operations in the same order), **distributed locks** and configuration stores (etcd, Consul, ZooKeeper). Whenever correctness demands that everyone agrees, consensus is the tool.

22.3 Majority quorums



A value is agreed once a **majority** ($N/2 + 1$) accepts it. This tolerates a minority failing — a 5-node cluster survives 2 failures, a 3-node cluster survives 1 (hence **odd** cluster sizes). Because any two majorities overlap in at least one node, two conflicting values can never both be committed.

22.4 Raft, the understandable algorithm

Raft was designed to be easy to understand. It is **leader-based**:

- **Leader election:** time is divided into *terms*; if followers hear nothing from a leader, a node becomes a candidate and requests votes. Winning a majority makes it leader.
- **Log replication:** clients send commands to the leader, which appends them to its log and replicates to followers. Once a **majority** has stored an entry, it's **committed** and applied. Followers stay in lock-step with the leader's log.

Safety comes from the majority rule and from only electing leaders whose logs are sufficiently up to date.

22.5 Paxos and real systems

Paxos is the original consensus algorithm — provably correct but famously hard to implement; **Multi-Paxos** optimizes it for a stream of decisions. In practice: **etcd** and **Consul** use Raft, **ZooKeeper** uses ZAB, and Google's **Spanner** uses Paxos. You rarely implement consensus yourself — you use one of these as a coordination service.

Common mistakes

WATCH OUT

- Using an **even** cluster size (no majority tie-break; wastes a node).
- Trying to hand-roll consensus instead of using etcd/ZooKeeper/Consul.
- Expecting consensus to be cheap — it needs a round trip to a majority per decision.
- Running consensus across high-latency regions and suffering slow agreement.

Interview questions

1. What is consensus, and where is it needed?
2. Why does a majority quorum guarantee agreement and tolerate failures?
3. How does Raft elect a leader and replicate a log?
4. Why odd cluster sizes? How many failures does a 5-node cluster tolerate?
5. Name systems built on consensus.

Hands-on exercises

1. Compute the majority and fault tolerance for clusters of 3, 5, and 7.
2. Explain why two majorities in a 5-node cluster must overlap.
3. Describe what happens in Raft when the leader crashes.
4. Pick a coordination service for a distributed lock and justify it.

Mini project

USE A REAL CONSENSUS STORE

Run a 3-node etcd (or ZooKeeper) cluster. Elect a leader, write a key, and read it from all nodes. Kill one node and confirm the cluster still serves (majority intact); kill a second and watch it lose quorum and stop accepting writes. You'll see majority fault tolerance first-hand.

| Large project (capstone continues)

YOUR SOCIAL PLATFORM — WHERE CONSENSUS HIDES

Identify where your system relies on consensus: electing the leader for each database shard (Chapter 16), storing cluster configuration and feature flags, and coordinating distributed locks. Decide on a coordination service and a cluster size, and note the latency cost of putting any consensus on a hot path.

| Chapter summary

- **Consensus** gets a group to agree on one value despite failures — the basis of leader election, replicated logs, and coordination.
- A **majority quorum** ($N/2+1$) guarantees agreement (majorities overlap) and tolerates a minority failing — use odd sizes.
- **Raft** (leader election + log replication) is the understandable standard; **Paxos** is the original.
- Use **etcd/ZooKeeper/Consul** rather than implementing consensus yourself.

CHAPTER 23

Time, Clocks & Ordering

Part IV · Distributed Systems Theory — ordering events without a global clock.

LEARNING OBJECTIVES

- Explain why physical clocks can't reliably order events across machines.
- Define the happens-before relation and concurrency.
- Explain Lamport timestamps and what they can and can't tell you.
- Explain vector clocks and when you need them.

REAL-WORLD ANALOGY — WITNESSES WITH UNSYNCED WATCHES

Two witnesses in different cities, each with a slightly wrong watch, can't settle which of two events happened first by comparing watch readings. But if one witness *told* the other about their event before the second happened, we know an order — because information flowed between them. Logical clocks capture exactly that kind of ordering.

23.1 The problem

There is **no global clock** (Chapter 19). Each machine's wall-clock **drifts**, and even with NTP synchronization they disagree by milliseconds — enough that comparing timestamps to order events across machines is unreliable and can be flat wrong. Yet ordering matters constantly (which write is newer? did the reply come after the comment?). We need **logical** ordering, not physical time.

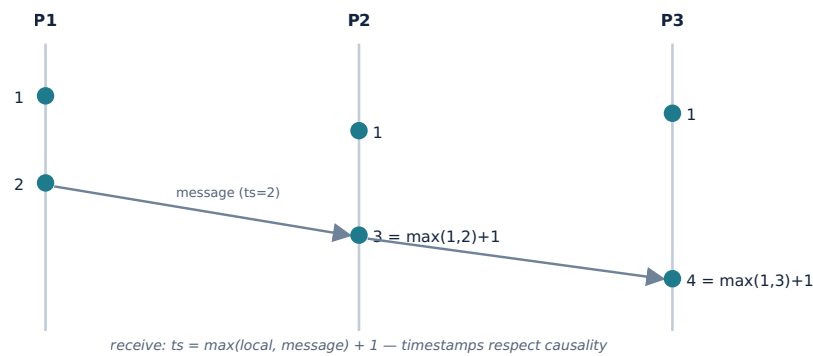
23.2 Physical clocks and NTP

Wall-clock time comes from each machine's quartz oscillator, which drifts. **NTP** periodically corrects it against reference servers, but leaves millisecond-to-tens-of-millisecond error, and clocks can even jump backward after a correction. Google's **TrueTime** is a rare exception: it exposes a bounded uncertainty interval (using atomic clocks and GPS) so Spanner can order events safely — most systems have no such luxury.

23.3 Happens-before

Leslie Lamport's **happens-before** relation (written $\rightarrow$) captures causality: if event A happens before B on the same node, or A is a message send and B its receipt, then $A \rightarrow B$. If neither $A \rightarrow B$ nor $B \rightarrow A$, the events are **concurrent** — their order is genuinely undefined.

23.4 Lamport timestamps



Each node keeps a counter, incremented on every event; a message carries its sender's timestamp, and the receiver sets its clock to $\max(\text{local}, \text{message}) + 1$. This gives a **total order consistent with causality**: if $A \rightarrow B$ then $\text{timestamp}(A) < \text{timestamp}(B)$. The catch: a smaller timestamp does *not* prove causality — Lamport clocks can't distinguish "caused" from "concurrent."

23.5 Vector clocks

A **vector clock** keeps one counter *per node*. By comparing whole vectors you can tell precisely whether two events are causally ordered or truly **concurrent** — which is exactly what you need to **detect conflicting writes** (Dynamo-style systems use them). The cost: the vector grows with the number of nodes, so it's heavier than a single Lamport counter.

Common mistakes

WATCH OUT

- Ordering events across machines by wall-clock timestamps and trusting the result.
- Assuming NTP makes clocks identical — it doesn't; clocks can even jump backward.
- Using Lamport timestamps to *detect* concurrency (they can't) — use vector clocks.
- Adding vector clocks everywhere and paying their size cost when you don't need conflict detection.

Interview questions

1. Why can't physical clocks reliably order distributed events?
2. Define happens-before and concurrency.
3. How do Lamport timestamps work, and what can't they tell you?
4. What do vector clocks add, and at what cost?

Hands-on exercises

1. Given three processes and two messages, assign Lamport timestamps to each event.
2. Give two events that are concurrent, and explain why.
3. Show a case where wall-clock ordering disagrees with causal ordering.

4. Explain how a vector clock detects a write conflict.

Mini project

IMPLEMENT LAMPORT AND VECTOR CLOCKS

Simulate three processes exchanging messages. Implement Lamport timestamps and verify that causally-ordered events always increase. Then implement vector clocks and detect a pair of concurrent (conflicting) writes that Lamport timestamps could not distinguish.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — ORDERING THAT MATTERS

Find where ordering matters: a comment and its reply must appear in causal order; two edits to the same field may conflict. Decide where you rely on causal ordering (Chapter 21) and where you'd use vector clocks to detect and resolve conflicting writes rather than trusting timestamps.

Chapter summary

- There is **no global clock**; physical clocks drift and NTP only approximates — don't order distributed events by timestamps.
- **Happens-before** ($\rightarrow$) captures causality; unrelated events are **concurrent**.
- **Lamport timestamps** give a total order consistent with causality but can't detect concurrency.
- **Vector clocks** detect causality vs concurrency precisely, at the cost of per-node counters.

CHAPTER 24

Distributed Transactions & Coordination

Part IV · Distributed Systems Theory — atomic commit across machines, and why we avoid it.

LEARNING OBJECTIVES

- Explain why atomic commit across machines is hard.
- Describe two-phase commit (2PC) and its blocking weakness.
- Explain why large systems avoid distributed transactions.
- Explain distributed locks and fencing tokens.

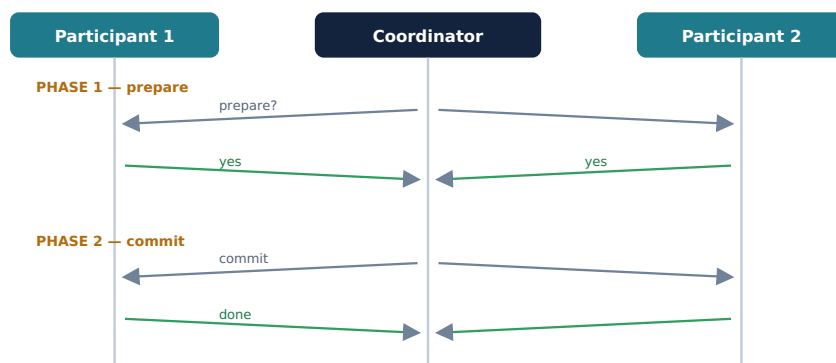
REAL-WORLD ANALOGY — BOOKING FLIGHT, HOTEL, AND CAR TOGETHER

A trip needs a flight, a hotel, and a car — booked through three separate agencies. You want all three or none. So you first ask each to *hold* a reservation; only if all three confirm do you tell them to finalize. If any can't, you cancel the holds. That two-step “hold, then confirm” is exactly two-phase commit — and you can already feel the risk: what if you faint between the steps, leaving three holds stuck?

24.1 The problem

Chapter 14 gave ACID on one machine. Across machines or services (after sharding, Chapter 17), a single logical operation may touch several nodes that must **all commit or all abort**. Partial failure (Chapter 19) makes this genuinely hard: any participant or the coordinator can crash at the worst moment.

24.2 Two-phase commit (2PC)



In **phase 1 (prepare)** the coordinator asks every participant if it can commit; each votes yes (and locks the data) or no. In **phase 2**, if *all* voted yes the coordinator tells everyone to commit; otherwise everyone aborts. This guarantees atomicity.

THE BLOCKING PROBLEM

2PC's fatal weakness: if the **coordinator crashes** after participants voted yes but before it says commit, those participants are **stuck** — holding locks, unable to decide, waiting indefinitely. 2PC is also synchronous and slow (multiple round trips plus held locks). **Three-phase commit (3PC)** adds a phase to reduce blocking but is rarely used and still fails under partitions.

24.3 Why big systems avoid distributed transactions

KEY INSIGHT — TRADE ATOMICITY FOR AVAILABILITY

Held locks, blocking, and a coordinator single point of failure make 2PC a poor fit for high-scale, highly-available systems. So large systems usually **avoid cross-service transactions** and instead use the **Saga** pattern (Chapter 38): break the operation into a sequence of local transactions, each with a **compensating action** to undo it, accepting eventual consistency. You give up instant atomicity to keep availability and scale.

24.4 Distributed locks and fencing

Sometimes nodes need **mutual exclusion** — only one may act at a time (a leader, a resource owner). A **distributed lock** from a coordination service (ZooKeeper, etcd — built on consensus, Chapter 22) provides this. But locks in distributed systems are tricky: a client can hold a lock, pause (GC or network stall), and act after its lock expired. The fix is a **fencing token** — a monotonically increasing number handed out with the lock, which the resource checks to reject a stale lock-holder's late writes.

Common mistakes

WATCH OUT

- Reaching for 2PC across services in a high-availability system — blocking and locks bite.
- Ignoring the coordinator-crash window that leaves participants stuck.
- Using a distributed lock without a **fencing token** and getting stale-holder writes.
- Assuming a lock service removes the need to design for failure.

Interview questions

1. Why is atomic commit across machines hard?
2. Walk through 2PC. What is its blocking problem?
3. Why do large systems prefer Sagas over distributed transactions?
4. What is a fencing token, and what problem does it solve?

Hands-on exercises

1. Diagram 2PC and mark the coordinator-crash window that causes blocking.
2. Rewrite a “transfer + notify” cross-service operation as a Saga with compensations.
3. Explain how a fencing token rejects a paused lock-holder's late write.

4. Decide: distributed transaction or Saga for placing an order across inventory + payment services.

Mini project

2PC VS SAGA

Model a two-service “order” operation (reserve inventory, charge payment). Implement it once with a simple 2PC-style coordinator and once as a Saga with compensating actions (release inventory, refund). Inject a failure after the first step in each and compare how they recover — blocked locks vs a clean compensation.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — CROSS-SERVICE OPERATIONS

List operations that span services (e.g. sign-up creating a user + provisioning a feed + sending a welcome message). For each, choose **Saga** over 2PC and define the compensating action for every step. Identify where you need a **distributed lock** (leader per shard) and specify a fencing token. This sets up the resilience patterns of Part VI.

Chapter summary

- Atomic commit across machines is hard because any node can fail mid-operation.
- **2PC** (prepare then commit) guarantees atomicity but **blocks** if the coordinator dies and holds locks; **3PC** helps little.
- Large systems avoid distributed transactions, preferring **Sagas** with compensations and eventual consistency.
- **Distributed locks** (via consensus services) need **fencing tokens** to be safe.

CHAPTER 25

REST & API Design

Part V · Communication & Architecture — the web's dominant API style.

LEARNING OBJECTIVES

- Explain REST's core principles: resources, HTTP verbs, statelessness, representations.
- Design clean resource URLs and use status codes correctly.
- Apply idempotency and safety to make APIs retry-friendly.
- Handle versioning, pagination, and filtering.
- Know REST's limits and what comes next.

REAL-WORLD ANALOGY — A STANDARDIZED ORDER FORM

A well-run catalog company gives every product a fixed catalog number and a single standard form with a few actions — view, add, change, remove. Anyone can order without special instructions because the structure is predictable. REST does this for software: every **thing** gets an address, and a small, fixed set of **verbs** acts on it.

25.1 Part V begins

Parts III–IV distributed and reasoned about data. Part V is about how clients and services **communicate**. We start with **REST**, the style that dominates web APIs, then cover gRPC, GraphQL, messaging, and the architectures built on them.

25.2 REST principles

REST models everything as **resources** (nouns) addressed by URLs, acted on by the standard **HTTP methods** (Chapter 4) as verbs. It is **stateless** (Chapters 4, 6) — each request carries all it needs — and exchanges **representations** (usually JSON). The interface is **uniform** and predictable.

Request	Meaning	Typical status
GET /users/42	Read user 42	200, or 404
GET /users/42/posts	List that user's posts	200
POST /posts	Create a post	201 Created
PUT /posts/9	Replace post 9	200 / 204
DELETE /posts/9	Delete post 9	204 No Content

25.3 Designing good resources

- Use **nouns, not verbs**: /users/42/posts, never /getUserPosts.
- Model **hierarchy** and use **plural** collection names.

- Return the **right status codes** (Chapter 4) — don't send `200` with an error body.
- Keep responses consistent and predictable in shape.

25.4 Idempotency and safety

KEY INSIGHT — DESIGN FOR RETRIES

Networks fail (Chapter 19), so clients retry. **GET** is safe (no effect), and **PUT/DELETE** are **idempotent** (Chapter 4) — doing them twice equals once, so retries are safe. Plain **POST** is not, so for critical creates accept an **idempotency key** the client sends, and de-duplicate on it. Retry-safety is a first-class API design concern, not an afterthought.

25.5 Versioning, pagination, filtering

- **Versioning:** evolve without breaking clients — e.g. `/v1/` in the path (simple and common) or a version header.
- **Pagination:** never return unbounded lists; prefer **cursor/keyset** pagination over large `OFFSET`s (Chapter 12).
- **Filtering & sorting:** use query parameters (`?status=active&sort=-created_at`).

25.6 Strengths and limits

REST is simple, uniform, cacheable (Chapter 8), and universally understood — an excellent default. Its limits appear at scale: clients often **over-fetch** (get more than needed) or **under-fetch** (need many calls), and mobile screens can require many round trips. Those pains motivate **gRPC** (Chapter 26), **GraphQL** (Chapter 27), and gateway **aggregation** (Chapter 9).

Common mistakes

WATCH OUT

- Verbs in URLs (`/createUser`) instead of resources plus methods.
- Returning `200 OK` with an error hidden in the body.
- Non-idempotent creates with no idempotency key, causing duplicates on retry.
- Unbounded list endpoints and giant `OFFSET` pagination.
- No versioning strategy, so any change breaks existing clients.

Interview questions

1. What are REST's core principles?
2. How do you design resource URLs and choose status codes?
3. Which methods are safe and idempotent, and why does it matter for retries?
4. How do you version and paginate an API?
5. What are REST's limitations, and what addresses them?

Hands-on exercises

1. Design REST endpoints for a blog (users, posts, comments) with methods and status codes.
2. Rewrite `/getUserPosts?id=42` and `/deletePost?id=9` as RESTful routes.
3. Add an idempotency key to a “create payment” endpoint.
4. Design cursor pagination for a comments list.

Mini project

DESIGN AND BUILD A SMALL REST API

Build a small REST API for a to-do or blog domain with proper resources, methods, and status codes. Add cursor pagination, a filter parameter, an idempotency key on creates, and a `/v1/` version prefix. Test retry behavior by sending the same create twice with the same key and confirming only one record appears.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE PUBLIC API

Design the REST surface for your platform: resources for users, posts, comments, follows, and likes, with correct methods, status codes, idempotency keys on creates, cursor pagination for feeds/comments, and a versioning scheme. Note which endpoints suffer over/under-fetching — candidates for GraphQL or gateway aggregation in the coming chapters.

Chapter summary

- **REST** models **resources** (nouns) acted on by HTTP **verbs**, stateless, exchanging JSON representations.
- Design clean noun-based URLs and return correct **status codes**.
- Use **safety and idempotency** (plus idempotency keys) to make APIs retry-friendly.
- Handle **versioning, cursor pagination, and filtering** deliberately.
- REST is a great default; over/under-fetching and round trips motivate gRPC, GraphQL, and aggregation.

COMING NEXT

Chapter 26 — RPC & gRPC. When services talk to *each other*, calling a remote function directly — with a compact binary format over HTTP/2 — often beats REST. We’ll cover RPC, Protocol Buffers, streaming, and when to choose gRPC over REST.